

1. Brief description of uploaded material

We provided the AI with our Lab 1 system description and class diagram for **Smart Commute Optimizer**, an application that recommends 2–3 commute routes based on user inputs (origin, destination, departure time) and preferences (e.g., fastest, most reliable, most comfortable). The design includes real-time transport data integration (e.g., MRT crowding/wait time, taxi availability, carpark availability), risk scoring (delay/crowding), and explainable route recommendations.

2. Prompt + AI response

Prompt:

Based on the uploaded system description and class diagram for our “Smart Commute Optimizer” (a Singapore commute recommender using real-time MRT crowding/wait time, taxi availability, and carpark availability), recommend a technology stack appropriate for an entry-level software engineering team. Prioritize simplicity and common industry tools, and explain why each component fits.

AI response (technology stack recommendation)

Recommended tech stack (entry-level, simple, industry-common)

1) Client (UI)

- **React Native + TypeScript (Expo)**
 - One codebase for iOS/Android, fast to build and maintain.
 - TypeScript reduces bugs and makes data models consistent with backend.
- **Maps SDK (Google Maps or Mapbox)**
 - Needed for origin/destination selection, map display, and route visualization.

2) Backend (API + business logic)

- **Node.js + TypeScript with NestJS**
 - Clean “Controller–Service–Provider” structure that matches your design (TripController, RouteScoringController, DataFetchController).
 - Easy REST APIs, good testing support, common in industry.

3) Database (persistent data)

- **PostgreSQL**
 - Reliable standard choice for user profiles, preference settings, trip requests/history, audit logs.

- **Prisma ORM**
 - Beginner-friendly migrations and strong TypeScript typing from DB to code.

4) Cache + real-time freshness handling

- **Redis**
 - Ideal for storing latest transport snapshots (MRT crowding/delay, taxi availability, carpark availability) with timestamps.
 - Enables fast responses and consistent “fallback to most recent cached data” when a real-time source fails.
 - Supports showing “last updated” and triggering warnings when data is stale.

5) Background jobs (data refresh)

- **BullMQ (Redis-backed)**
 - Simple scheduled refresh every few minutes + on-demand refresh when user taps “Refresh Routes”.
 - Keeps external API calls off the request path to meet performance targets.

6) Testing

- **Jest (unit tests)**
 - Focus on the most critical rules: freshness checks, fallback consistency, scoring and ranking.
- **Supertest (API integration tests)**
 - Verifies endpoints and end-to-end behavior quickly.

7) Observability (debugging + reliability)

- **Structured logging (Pino)**
 - Easy to trace failures from external data sources and scoring decisions.
- **Error tracking (Sentry)**
 - Captures crashes in both client and server with stack traces, good for small teams.

8) DevOps / Deployment

- **Docker**
 - Consistent local dev and deployment environments.
- **GitHub Actions (CI)**
 - Automated lint/test/build on every push.
- **Simple hosting**
 - Start with a single small deployment (API + Redis + Postgres) to reduce ops overhead; scale later if needed.

3. Opinion on one non-language technology

Chosen technology:

Chosen technology: Redis

Redis is appropriate for our project because our system depends heavily on frequently updated transport data (e.g., MRT crowding and taxi availability), and caching the latest snapshots helps ensure fast response times while supporting fallback to the most recent valid data when APIs fail. For a course project, Redis adds meaningful architectural realism without excessive complexity, since it integrates cleanly with Node/NestJS and supports both caching and background job queues (via BullMQ). However, we would keep the deployment simple (single instance, no clustering) to avoid unnecessary operational overhead.